Practical Application of Agile PM Principles

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

This slide contains material that is copied from the book Agile Project Manager_v2_handbook by Agile Business Consortium. All rights reserved. No part of this material may be reproduced or transmitted in any form or by any means without the prior written permission of the copyright owner. This material is used here for educational purposes under the doctrine of fair use, with the intention to support learning and teaching.

For more information, please refer to the original source:
Book Title: Agile Project Manager_v2_handbook
Author: Agile Business Consortium
Publisher: Buckland Media Group Limited
Year of Publication: 2017
Please consult the book for further details and complete content.

innovative • entrepreneurial • global

Outline

- DSDM Practice : MoSCoW Prioritization
  - Introduction
  - The MoSCoW Rules
  - MoSCow relating to a Specific Timeframe

- DSDM Practice : Timeboxing
  - Timeboxing Options
  - The Daily Stand Up (Scrum)
  - Timeboxes - The Wider Context

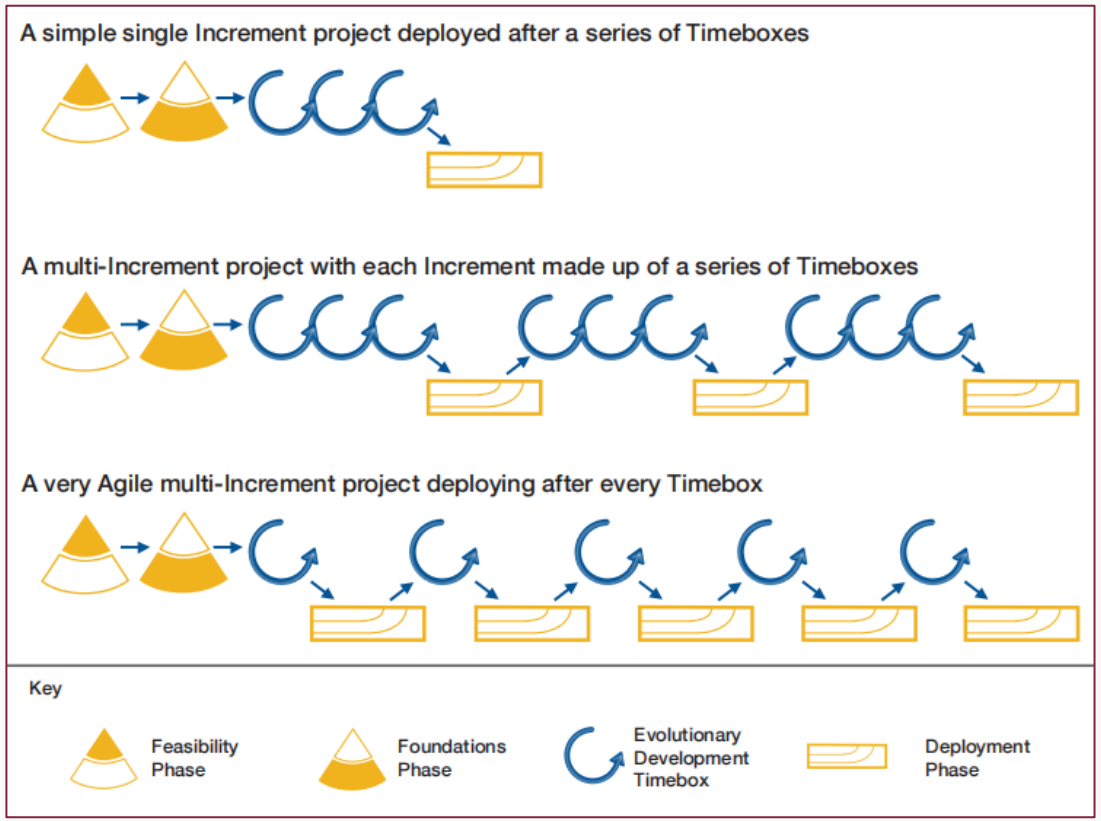
DSDM Practice : MoSCoW Prioritisation

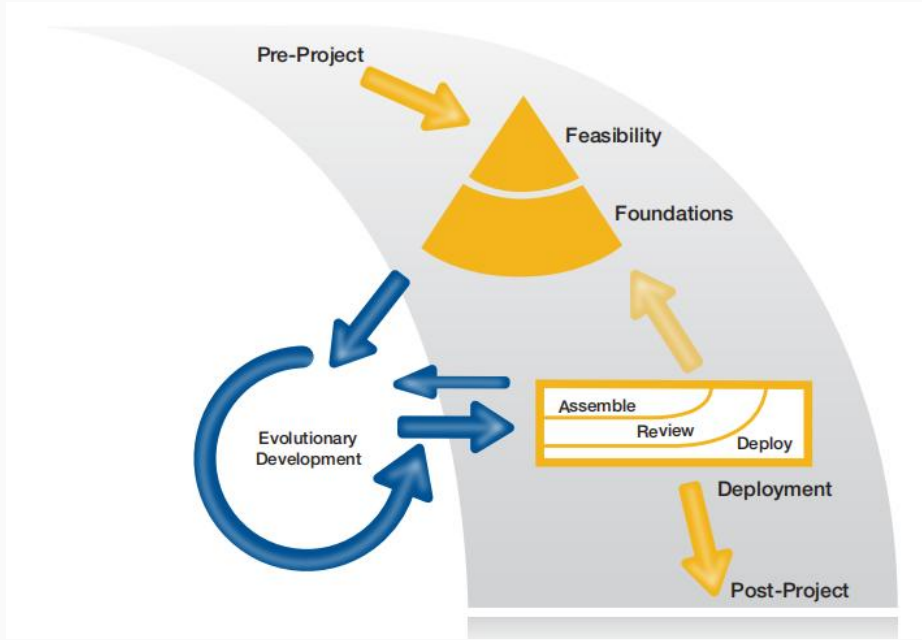
UTM
UNIVERSITI TEKNOLOGI MALAYSIA

innovative • entrepreneurial • global

Recap - Configuring the Process - 3 lifecycle examples

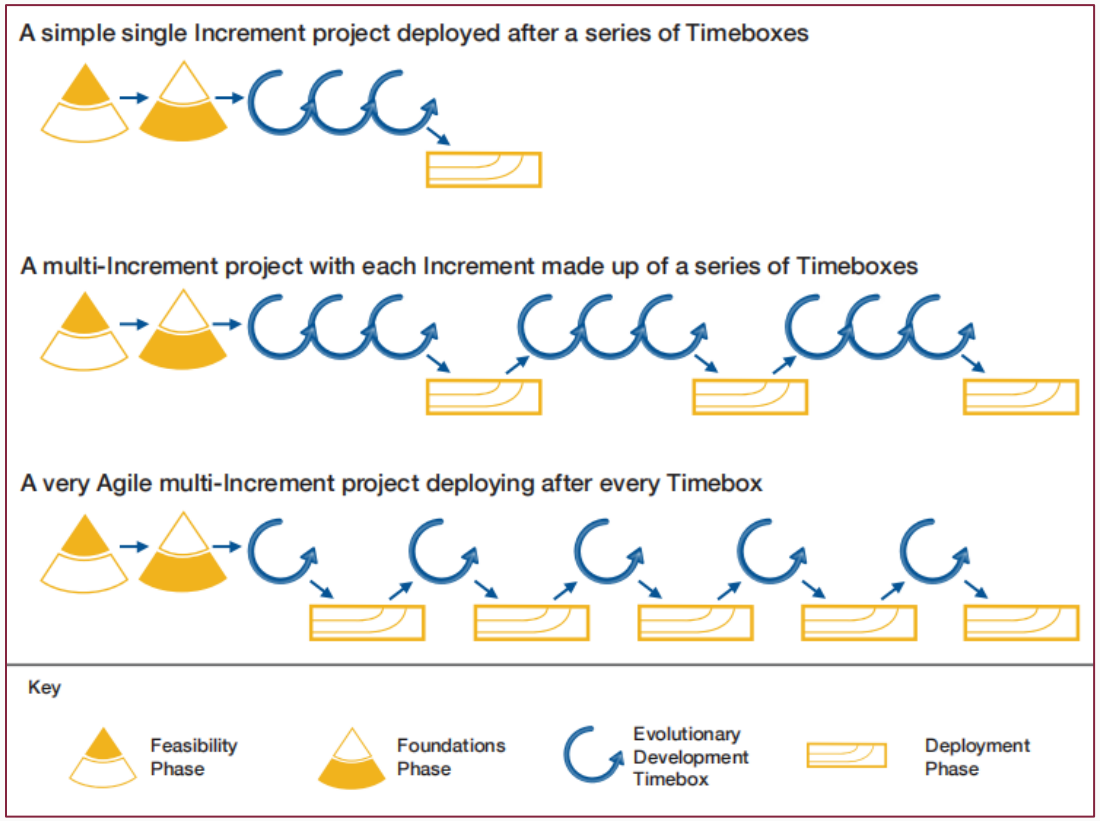
- Solution built and deployed incrementally from a firm foundation
- Ensure value delivered early and delivered frequently





3 Options lifecycle

DSDM Lifecycle Option	Description / Implementation Approach	Example of How Phases Are Organized
1. Single-Increment Project	• The simplest form of DSDM implementation.• The project goes through Feasibility and Foundations once, then proceeds through several Timeboxes (iterations) in the Evolutionary Development Phase before a single Deployment at the end.• Suitable for smaller, well-defined projects with stable requirements.	**Sequence:**Feasibility → Foundations → (Multiple Evolutionary Development Timeboxes) → Single Deployment → Post-Project.**Example:** A small internal HR application built and deployed once after full iteration cycles.
2. Multi-Increment Project	• The project is divided into multiple Increments, each made up of several Timeboxes.• Each Increment represents a partial but usable version of the overall solution.• After each Increment, a Deployment occurs, followed by the next Increment cycle.• Suitable for medium-to-large projects where early releases deliver business value progressively.	**Sequence:**Feasibility → Foundations → Increment 1 (several Timeboxes → Deployment) → Increment 2 (several Timeboxes → Deployment) → Increment 3 (several Timeboxes → Deployment) → Post-Project.**Example:** An e-learning platform where each Increment adds a new module (e.g., user management, course upload, quiz engine).
3. Very Agile Multi-Increment Project	• The most agile configuration of DSDM.• Deployment occurs after every Timebox instead of after a series of Timeboxes.• Ensures very frequent delivery, tight feedback loops, and rapid adaptation to change.• Suitable for dynamic environments or continuous delivery setups.	**Sequence:**Feasibility → Foundations → (Timebox → Deployment → Timebox → Deployment ...) → Post-Project.**Example:** A SaaS web application that releases updates or features after every sprint (e.g., new dashboard, bug fixes, small enhancements).



DSDM Phases, Key Products, and Roles Involved (with Evolutionary Phase)

DSDM Phase	Key DSDM Products (Deliverables)	Roles & Their Description / Focus in This Phase
1. Pre-Project	• Terms of Reference (ToR)	**Business Sponsor** – Approves project concept and initial funding. **Business Visionary** – Outlines desired outcomes and business drivers. **Project Manager** – Plans initial setup, identifies stakeholders. **Technical Coordinator** – Advises on initial feasibility and technology suitability.
2. Feasibility Phase	• Feasibility Assessment • Feasibility Prototype (optional)	**Business Analyst** – Defines outline requirements and assesses viability. **Technical Coordinator** – Evaluates technical feasibility and constraints. **Business Visionary** – Confirms business alignment. **Project Manager** – Coordinates feasibility study, estimates scope, time, and risk.
3. Foundations Phase	• Business Case • Business Vision • Prioritised Requirements List (PRL) • Solution Architecture Definition (SAD) • Management Approach Definition (MAD) • Development Approach Definition (DAD) • Delivery Plan	**Business Visionary** – Defines vision and validates alignment with goals. **Business Sponsor** – Approves the Business Case and baseline scope. **Project Manager** – Leads planning, governance, and ensures consensus. **Business Analyst** – Develops PRL and ensures traceability. **Technical Coordinator** – Designs solution architecture and standards. **Business Ambassador** – Validates business priorities and feasibility.

DSDM Phases, Key Products, and Roles Involved (with Evolutionary Phase)

4. Evolutionary Phase *(combines Exploration + Engineering)*	• Timebox Plan(s) • Evolving Solution • Solution Increment(s) • Daily/Timebox Review Records • Release Plan	**Team Leader** – Facilitates team collaboration and delivery focus. **Business Ambassador** – Provides business input, validates requirements, and accepts deliverables. **Business Analyst** – Supports communication between business and developers, manages evolving requirements. **Solution Developer** – Designs, builds, and integrates solution components. **Solution Tester** – Verifies each increment within the timebox. **Technical Coordinator** – Ensures architecture integrity and standards. **Project Manager** – Monitors progress, manages risks, and supports replanning as needed.
5. Deployment Phase	• Deployed Solution • Benefits Assessment • Project Review Report	**Project Manager** – Oversees release and handover to business. **Business Visionary** – Confirms delivery meets business objectives. **Business Ambassador** – Supports acceptance and user transition. **Technical Coordinator** – Oversees deployment and ensures operational readiness. **Solution Developer/Tester** – Provides release and technical support.
6. Post-Project	• Benefits Realisation Review	**Business Sponsor** – Confirms achievement of planned benefits. **Business Visionary** – Measures long-term impact and value. **Project Manager** – Conducts post-implementation review. **Business Analyst** – Documents lessons learned and improvement opportunities.

DSDM Phases — Typical Challenges and How Products Solve

DSDM Phase	Possible Key Challenges	Related Product(s)	How the Product Solves / Clarifies the Challenge
1. Pre-Project	• Lack of clarity on project purpose or business justification. • Unclear sponsorship or funding commitment. • Misalignment of expectations between stakeholders.	**Terms of Reference (ToR)**	Summarises the project rationale, objectives, and expected business benefits. Establishes initial scope and confirms sponsor commitment, ensuring everyone starts with a common understanding.
2. Feasibility Phase	• Uncertainty whether the project is viable or worth pursuing. • Lack of understanding of risks, constraints, or dependencies. • Misalignment between technical feasibility and business goals.	**Feasibility Assessment** **Feasibility Prototype (optional)**	Feasibility Assessment analyses business, technical, and financial viability. Prototype provides tangible proof of concept to validate assumptions early and avoid waste.
3. Foundations Phase	• Misalignment between business and technical understanding. • Undefined scope or unclear priorities. • Unclear governance and approval process for changes. • Architecture not linked to business goals.	**Business Case** – clarifies justification and benefits. **Business Vision** – defines end-goal and desired outcomes. **Prioritised Requirements List (PRL)** – lists and ranks requirements (MoSCoW). **Solution Architecture Definition (SAD)** – aligns design with business need. **Management Approach Definition (MAD)** – defines roles, decision process, and responsibilities. **Development Approach Definition (DAD)** – outlines technical and build approach. **Delivery Plan** – sets high-level delivery roadmap.	Together, these products establish a clear baseline for scope, roles, governance, and architecture — removing ambiguity before development begins. They ensure both business and technical teams share the same foundation.

DSDM Phases — Typical Challenges and How Products Solve

4. Evolutionary Development Phase *(combining Exploration & Engineering)*	• Ambiguity in interpreting requirements. • Miscommunication between business and developers. • Frequent requirement changes without control. • Limited user feedback or late validation. • Integration or performance issues detected late. • Incomplete test coverage or lack of deployment readiness.	**Timebox Plan** – defines short-term deliverables and priorities. **Evolving Solution** – continuously tested and refined prototype. **Solution Increment** – provides deployable pieces of the system. **Release Plan** – coordinates delivery to business. **Daily/Timebox Review Records** – document feedback and agreement.	These products ensure iterative, collaborative progress. Regular reviews and MoSCoW prioritisation allow flexibility and early feedback. Continuous testing and integration reduce risk of late defects, while the Release Plan ensures readiness for deployment.
5. Deployment Phase	• Resistance to change during rollout. • Unclear handover or ownership. • Benefits not measured or reported. • Lessons not captured for improvement.	**Deployed Solution** – formal delivery and acceptance. **Benefits Assessment** – measures outcomes against Business Case. **Project Review Report** – documents lessons learned.	Ensures solution acceptance, verifies expected value delivery, and captures organisational learning for future projects.
6. Post-Project	• Benefits not tracked after delivery. • Lack of accountability for realised value. • Missed opportunities for process improvement.	**Benefits Realisation Review**	Confirms actual vs. planned benefits, validates ROI, and records lessons to guide future initiatives. Keeps sponsors accountable for achieving long-term outcomes.

DSDM Practice - MoSCoW Prioritisation

MoSCoW Prioritisation is a technique used to rank requirements by importance in Agile projects. It classifies them into four categories:

Must have = **Minimum Usable SubseT (Guaranteed)**
Without this requirement . no point on this date or not legal or unsafe or not a viable solution

Should have — Important but not vital
May be painful to leave out but solution still viable
May need workaround, which may be temporary

Could have — Wanted or desirable but less important
Less impact if left out (compared with a Should)

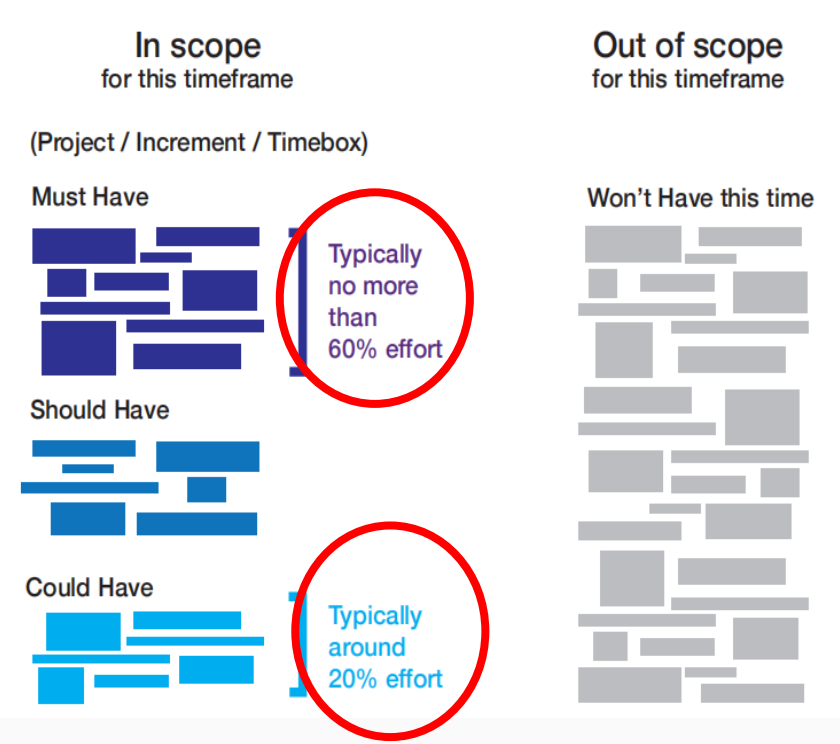
Won't have this time
Will not be delivered in this timeframe



MoSCoW Relating to a Specific Timeframe

This diagram shows how **MoSCoW prioritisation** allocates effort within a project or timebox:

- **Must Have (≤60% effort):** Critical features that must be delivered for the solution to work.
- **Should Have:** Important but not vital; can be postponed if needed.
- **Could Have (~20% effort):** Nice-to-have features that provide added value if time allows.
- **Won't Have this time:** Agreed as out of scope for the current timeframe, possibly for future releases.



It ensures focus on essentials while allowing flexibility for change and risk management.

Example: MoSCoW Effort Allocation in DSDM (Foundations Phase)

MoSCoW Category	Description	% of Total Effort (Typical in DSDM)	No. of Requirements (out of 100)	Estimated Hours Required	Estimated Days Required (at 24h/day)
Must Have	Essential for the solution to work; cannot drop	60%	60	0.60 × 960 = **576 hours**	576 ÷ 24 = **24 days**
Should Have	Important but not critical; may defer if time limited	20%	20	0.20 × 960 = **192 hours**	192 ÷ 24 = **8 days**
Could Have	Nice-to-have; used to absorb time variation	20%	20	0.20 × 960 = **192 hours**	192 ÷ 24 = **8 days**
Won't Have (this time)	Out of scope for this timeframe	–	0	–	–

Assume, 960 man-hours needed for this project, calculate how many days is needed?

Making MoSCoW Work

- Agree up-front how priorities will work

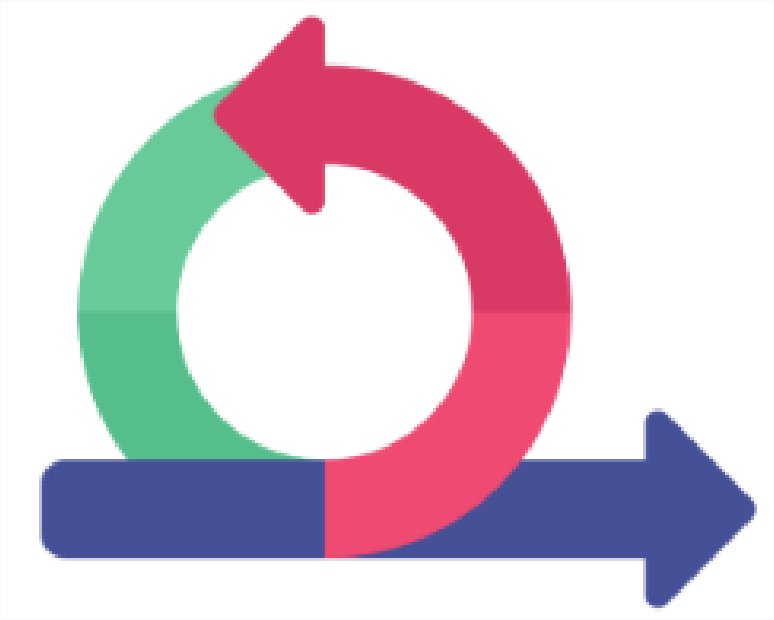
  - Must Haves non-negotiable

  - Should Haves and Could Haves are subjective

  - Discussion facilitated by the Project Manager and Business Analyst

- Majority of prioritisation starts at Foundations but priorities continually reviewed - they change over time
- When new requirements added, check impact

  - On project and on balance of MoSCoW effort



Making MoSCoW Work

- At end of Increment, all requirements not met are re-prioritised according to the need of the next Increment
- Use MoSCoW to manage business expectations

  - High % of Should Haves and Could Haves provide optimum contingency to ensure delivery of Must Haves
- PM responsible for ensuring Delivery Plan is realistic and achievable

More Example

Table 1: MoSCoW Allocation Across 4 Timeboxes (All Completed by TB4)

Timebox	Requirements Planned	Must Have	Should Have	Could Have	Key Notes
Foundations (Baseline PRL)	100	60	20	20	Priorities agreed and effort estimated using MoSCoW (960 hrs total).
Timebox 1	25	15	6	4	Core setup; focus on essential features to establish base functionality.
Timebox 2	25	12	8	5	Reprioritisation occurs — some "Should" items moved to TB3 due to time limits.
Timebox 3	25	10	9	6	Balance adjusted — "Could" items dropped temporarily to ensure "Should" items progress.
Timebox 4	25	8	10	7	New requirements added and reprioritised; all remaining items completed for deployment.
Total (End of TB4)	100	45 + 15 (TB4) = 60	33 + 7 (TB4) = 40	15 + 5 (TB4) = 20	**All requirements completed; system ready for release.**

More Example, helpful to plan TB

Table: Estimated Working Days per MoSCoW Category (within 78 days total)

Timebox	Requirements (Total)	Must Have	Est. Days (Must)	Should Have	Est. Days (Should)	Could Have	Est. Days (Could)	Total TB Days	Key Focus
TB1	25	15	15 days	6	3 days	4	2 days	20 days	Core setup; essential base
TB2	25	12	10 days	8	6 days	5	4 days	20 days	Reprioritisation; some "Should" deferred
TB3	25	10	8 days	9	7 days	6	5 days	20 days	Balance; ensure "Should" items progress
TB4	25	8	7 days	10	8 days	7	5 days	20 days	Final refinement; all completed
Total (End of TB4)	100	45 + 15 = 60	40 days	33 + 7 = 40	24 days	15 + 5 = 20	16 days	≈ 78 days	System ready for release

This simulation assumes one person-day of effort. If two developers are assigned, can the project duration be shortened?

More Example

Table 2: Example of Reprioritisation Scenarios Between Timeboxes

Scenario	Description	Example from HIP 2.0 Project Context	Result / Action Taken
1. Review priorities – some "Should" deferred	When schedule tight, non-critical "Should" features postponed to next TB.	"Automated supplier alert system" deferred from TB2 → TB3.	Time saved ensures completion of Must Have "Supplier registration validation".
2. Rebalance effort – Could Haves used for flexibility	If progress faster than expected, use remaining time to include some "Could" features.	"Dashboard colour themes" (Could Have) added back in TB3.	Increases user satisfaction without impacting deadlines.
3. New requirements reprioritised – final refinements	New requests or feedback introduced late; reprioritised against existing MoSCoW list.	New "Halal certificate renewal reminder" added in TB4 after pilot feedback.	Replaced low-value Could Have "Export summary chart" in final delivery.

Prioritisation - Who Decides?

- Are all Must Haves non-negotiable?
  - Must Have = "Deliver this or we cancel the project
  - PM or Business Analyst may challenge less obvious Must Haves
  - Can this requirement be broken down further?
    - A high-level Must may contain a mix of Musts, Shoulds, Coulds, Won'ts
- Business Visionary and Ambassador have final say
- The Business Sponsor perspective
  - Expects all Must Haves
  - Typically also expects most / all Should Haves to be delivered
  - DSDM's recommended % split gives at least 20% "contingency" for Could Haves



Note: 10% contingency is the normal contingency on traditional projects

Assigning Priorities - AgilePM Top Tips

- Ensure business roles understand why and how DSDM priorities requirements
- Consider starting all requirements as Won't Have, then justify why priority is higher
- Question Must Haves

  - "If on night before Deployment, this requirement is not ready, would you cancel project?"

  - "Is there a workaround, even if it is painful?" If Yes, then it is not a Must
- Ask why this requirement is needed? For Project? For Increment?

Assigning Priorities - AgilePM Top Tips

- Is this requirement dependent on completion of other requirements?

  - A Must cannot depend on a Should Have, Could Have or Won't Have

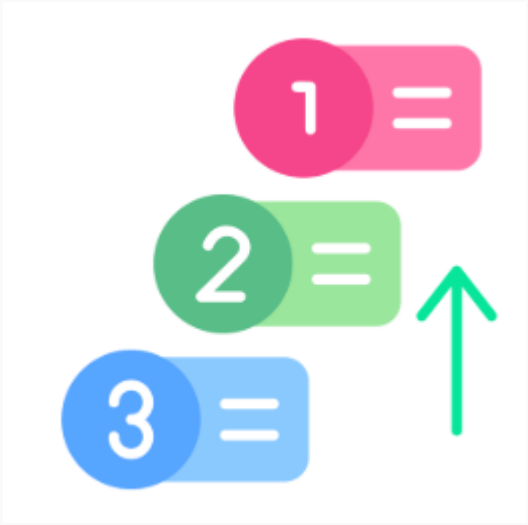
- Allow for different priorities of acceptance criteria
- Can this requirement be decomposed?

  Decomposed elements may have different priorities

- Tie requirements to a project objective :

  If objective not a Must, then requirement cannot be a Must

- MoSCoW helps prevent effort spent on "interesting" rather than "important" things



More example

Table: AgilePM Top Tips for Assigning Priorities (MoSCoW in DSDM)

AgilePM Tip	Explanation	Example (HIP 2.0 Project)
1. Ensure business roles understand why and how DSDM prioritises requirements	Everyone must agree on the importance of prioritisation and its impact on delivery. PM and BA facilitate this understanding.	The Business Ambassador and Visionary agree that only "Must Have" features go live first to ensure the platform functions correctly.
2. Start all requirements as 'Won't Have', then justify why they're higher	Encourages strong reasoning before upgrading an item to Must/Should/Could; prevents overloading the plan.	Start all 100 HIP requirements as Won't Have; "User registration" is justified as Must because system cannot function without it.
3. Question all Must Haves	Test each "Must" using these questions: • *If not delivered, would we cancel the project?* • *Is there a workaround?* If yes, it's not a Must.	"Payment gateway integration" has a manual workaround via bank transfer — hence reclassified from Must to Should.
4. Ask why the requirement is needed (Project or Increment?)	Clarifies timing and purpose — some features may be more relevant later.	"Supplier performance analytics" needed only in Phase 2 → deferred to next increment.
5. Check dependency on other requirements	A Must cannot depend on a Should/Could/Won't Have. Dependencies must align with priorities.	"Report export" (Should) depends on "Dashboard module" (Must), so dashboard must be completed first.
6. Allow for different priorities in acceptance criteria	A requirement can have sub-parts with varied priorities.	"User login" → Must: basic authentication; Should: 2FA; Could: biometric login.

More example

7. Decompose high-level requirements	Break large Musts into smaller items which can have different priorities.	"Supplier management" split into "Add supplier" (Must), "Edit supplier" (Should), "Archive supplier" (Could).
8. Tie requirements to project objectives	If a requirement doesn't directly support a project goal, it shouldn't be a Must.	"Halal certificate validation" directly supports HIP 2.0's compliance goal → Must Have.
9. Use MoSCoW to manage business expectations	Reminds stakeholders that not everything can be delivered immediately; supports realistic planning.	Business Sponsor expects 100% delivery but PM reminds that Could Haves act as contingency.
10. MoSCoW helps avoid spending effort on 'interesting' instead of 'important' items	Keeps focus on business value, not personal preferences.	"Animated homepage banner" dropped (interesting) to focus on "Audit report generator" (important).

DSDM Practice : Timeboxing

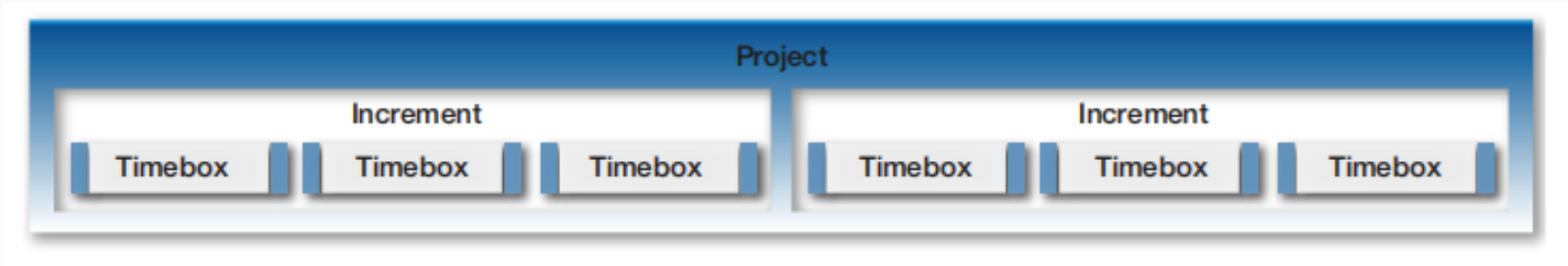
UTM
UNIVERSITI TEKNOLOGI MALAYSIA

innovative • entrepreneurial • global

Timeboxes - How Long?

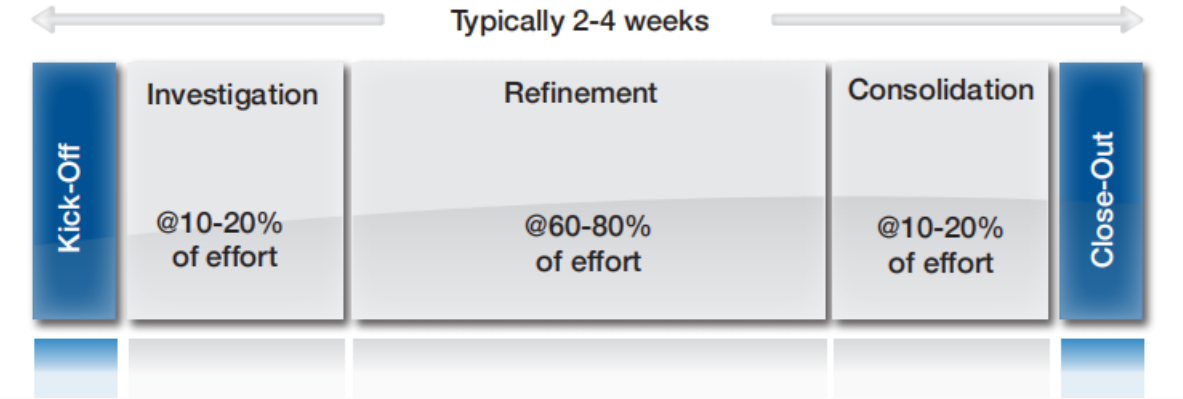
During Foundations, Agile PM helps Solution Development Team agree preferred length of Timebox

- Short enough to keep team focused
- Long enough to achieve something complete and meaningful
- Usually between 2 and 4 weeks
- Longer Timeboxes should be avoided unless really necessary



Timeboxing - Which style?

- DSDM Structured Timebox



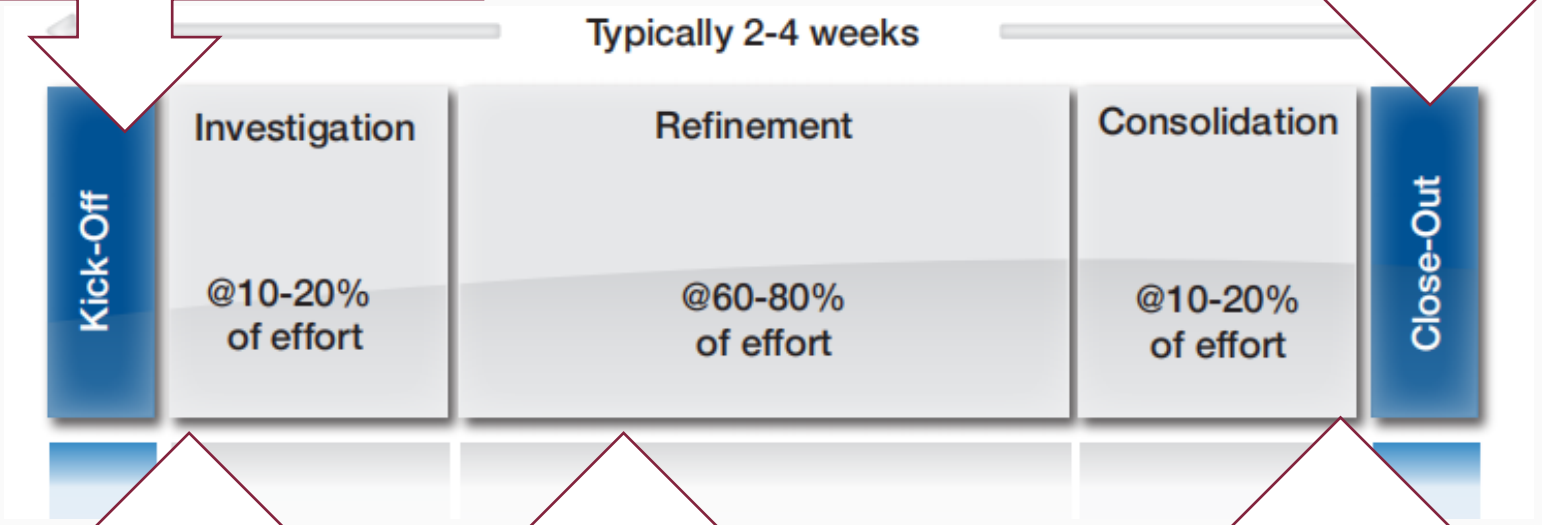
- Free Format Timebox



A DSDM Structured Timebox

- Check TB objectives in Delivery Plan still feasible
- Agree acceptance criteria, where possible
- Review availability of SDT members
- Highlight known dependencies

- Sign-off what has been delivered
- Assess impact of what has not been "done"
- Build on good experience
- Avoid repeating same mistakes
- Define issues to be resolved for next TB



- Understand detail and priorities of work to be done
- Review dependencies, Timebox Plan and risks

- Work on the Solution in line with agreed MoSCoW priorities
- Review at end of Refinement (work should be nearly ready)

- Complete actions from Refinement review
- Complete final testing
- Ensure organisational or project standards have been met, with any final quality checks
- Review at end of Consolidation, ensuring TB objectives are met.
- Any products not meeting acceptance criteria deemed as "Not Delivered"

Timeboxing - Which style?



- Reflects style used by other popular Agile approaches e.g. Scrum Sprint
- Used when DSDM Structured Timebox not possible or not helpful
- Starts with Kick-off and finishes with Close-out
- In between, any number of formal or informal review points
- SDT pick up one or more products or requirements/User Stories and evolve these iteratively until completed and meet agreed acceptance criteria
- SDT then pick up the next product or requirement/User Story and repeat the process
- Free format style relies on Business Ambassador being available consistently to review and provide feedback on ongoing basis

Timeboxing - Agile PM Top Tips

- Ask "How soon can Tester start testing?" and "When is the earliest we can complete a User Story?" Early completion (within a few days) demonstrates tangible progress
- Believe in the TB : encourage SDT to finish on time by prioritising work to be done
- Never let TB "overrun"
- Insist works stops at agreed time and work out what is "done"
- Even if TB has correct balance of priorities, individual skills and capabilities of SDT may still make achieving objectives difficult / impossible

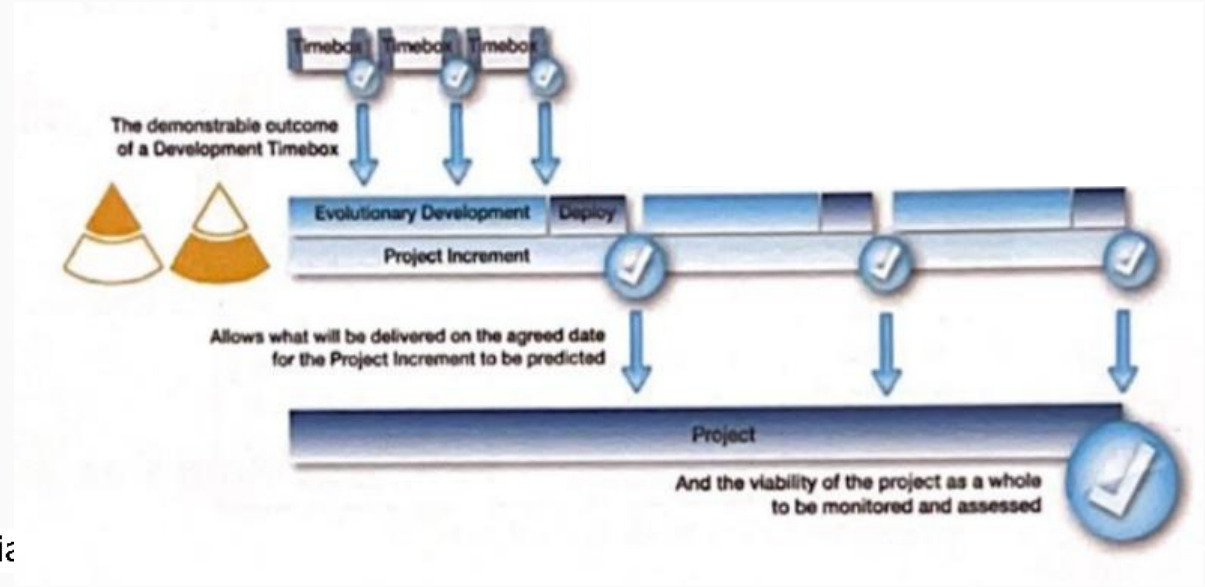
Timeboxing - Agile PM Top Tips

- Check TB progress by observing daily Stand-Ups and enabling SDT to react quickly to problems raised
- Build environment of trust and no-blame, to ensure SDT are open about progress
- Ensure TB Kick-off involves right people
- Ensure acceptance criteria for successful TB completion are clear at Kick-off
- Ensure key risks affecting TB are identified at Kick-off, and monitored during TB
- Ensure Close-out performs a Retrospective to feed learnings into next TBs



Timeboxing - Which style?

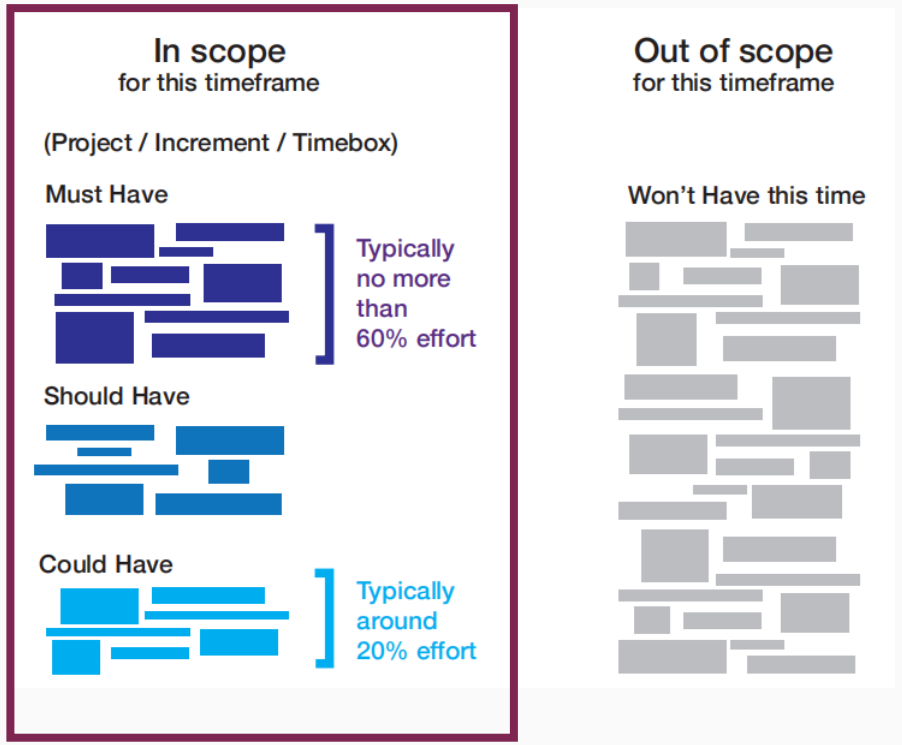
- Combining Timeboxing with MoSCoW enables consistent and predictable on-time delivery of projects
- Project and Increment delivery founded on delivery of Development TBs
- Development Timeboxes:
  - Provide control at lowest level - on-going validation of Delivery Plan
  - Provide clear indication of project health
  - Prove estimating is working (or quickly highlight if not working)
  - Prove SDT is working effectively (or quickly highlight if not effective)
- Confidence feeds upwards to Increment and Project levels



MoSCoW - Integral Part of Planning Process

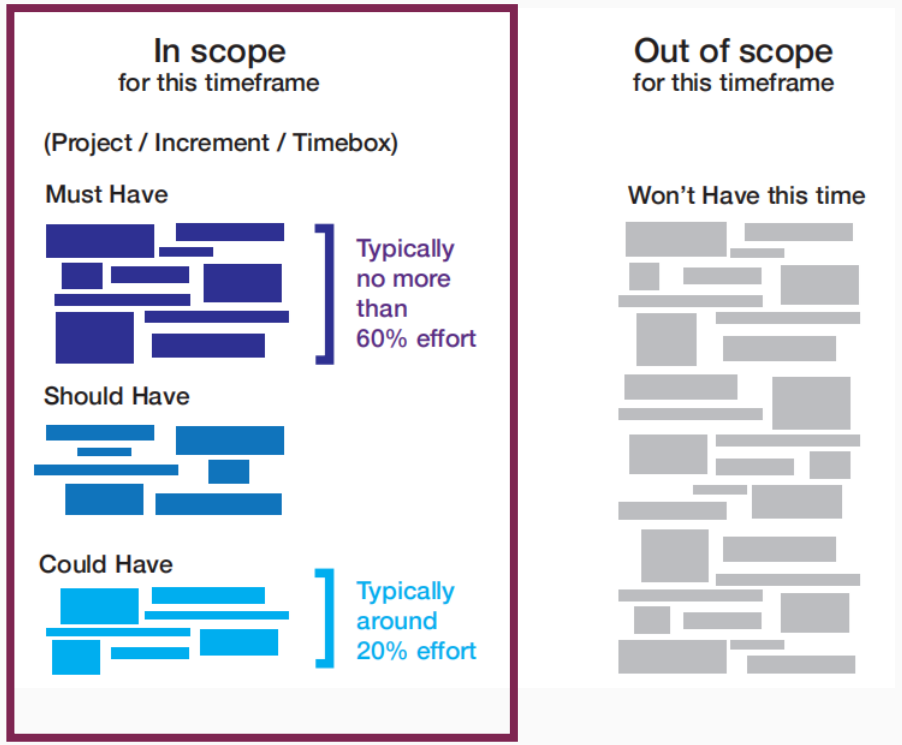
Getting sensible MoSCoW balance at Project and Increment levels achieved through collaboration between whole team

- Business roles work together
  - To ensure priorities reflect true business needs
  - To group requirements to provide effective and valuable solution to meet business deadlines
- Solution / Technical roles work together
  - To ensure plans to deliver these priorities are realistic, achievable reflect any technical constraints and are being planned in most efficient way



MoSCoW - Integral Part of Planning Process

- Business Analyst
    - Considers links between requirements and availability of information
        - Helps to identify viable groups of requirements
        - Agile Project Manager
        - Ensures balance of priorities reflects level of known risks on this specific project



The Daily Stand-up and the AgilePM

- Opportunity for Solution Development Team
  - To share information
  - To carry out day-day re-planning and re-organising
- This is in addition to daily ongoing informal SDT communication
- Agile PM regularly attends, primarily as observer
  - To understand progress and pick up on escalated issues
  - But always remember Stand-up is run by SDT and for SDT
    - They are not "Reporting" to PM
- Visionary and Technical Coordinator also observe to keep in touch with progress
  - Visionary is also providing visible ongoing support
  - Technical Coordinator is keeping up with technical decisions and picking up escalated technical issues

The Daily Stand-up and the AgilePM

- A key part of all Timeboxes
- Takes place same time, same place every day
- SDT share information to enable any day-to-day replanning and reorganising
- Simple format
  1. What I have done since the last stand-up to help achieve the Timebox objectives?
  2. What I will be doing until the next stand-up to help achieve the Timebox objectives?
  3. What problems, risks or issues (blockers) do I have that will prevent me or the team achieving the Timebox objectives?
- As a guide, allow 2 minutes per person + 2 minutes

Empowered Teams and Change in Timeboxes

- SDT must be appropriately empowered to handle any change falling within agreed TB objectives without formal change control
  - i.e. without needing to go outside SDT
- As a guide, following scenarios mean change of scope and therefore need to be handled more formally (outside the SDT)
  - Changing breadth of solution
    - Adding to high level requirements
    - Removing Must Have requirements (Increment or Project Level)
  - Increasing % of Must Haves
    - By adding new Musts or upgrading Won'ts, Coulds or Shoulds to Must Have
- SDT always empowered to operate within previously agreed boundaries without need to escalate to PM



THANK YOU

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

In the Name of God for Mankind

www.utm.my